Tic-Tac-Toe and Connect 4 Project

An additional participation opportunity this semester comes in the form of the semester-long Tic-Tac-Toe and Connect 4 project. There are five phases to this project; the first phase (Tic-Tac-Toe) is worth up to 5 participation points, while each of the other four phases (four different variants on Connect 4) is worth 15 participation points. An additional 25 points are available—5 for each phase—for the top 10 finishers in a tournament among all students who participate in the project.

The Games

In this series of participation projects, you will implement agents to play two games: Tic-Tac-Toe and Connect 4. There will be four different variations of Connect 4. You'll likely find, though, that your agent for each variation serves as a strong foundation for the next variation. While you can write a single agent to run both types of games, there is no restriction on just submitting to one or more with a specific agent focused on the particular type of game. You may submit to all 5 games or just one or two (the choice is yours as these are all optional projects).

Tic-Tac-Toe

The Game of Tic-Tac-Toe is a basic two player game consisting of a 3 x 3 grid. Each player takes turns selecting a position to place a game token (either an X or an O). The objective of the game is to get 3 tokens in a row either horizontally, vertically or diagonally.

Tic-Tac-Toe is a simple game that we learn as a child (sometimes one of the first intelligence games we learn). The game of Tic-Tac-Toe is considered a "Solved Game", a game where there exists an optimal strategy that guarantees the player will not lose. Implementing the optimal strategy is not trivial, however.

Connect Four

The Connect Four Game is also a basic two-player board game. The game board consists of a grid of six (6) rows of seven (7) columns. Each player takes a turn by dropping a player’s token into one of the 7 columns, and the players’ token will be dropped in and occupy the lowest row in that column that does not already contain a token (either your own or that of your opponent(s)). The object of the game is to connect four of your tokens in consecutive order while preventing your opponent from doing the same. To win the game, tokens could occupy 4 consecutive locations either horizontally, vertically or diagonally (either negative or positive slopes). See Figure 1 for examples of game wining combinations.

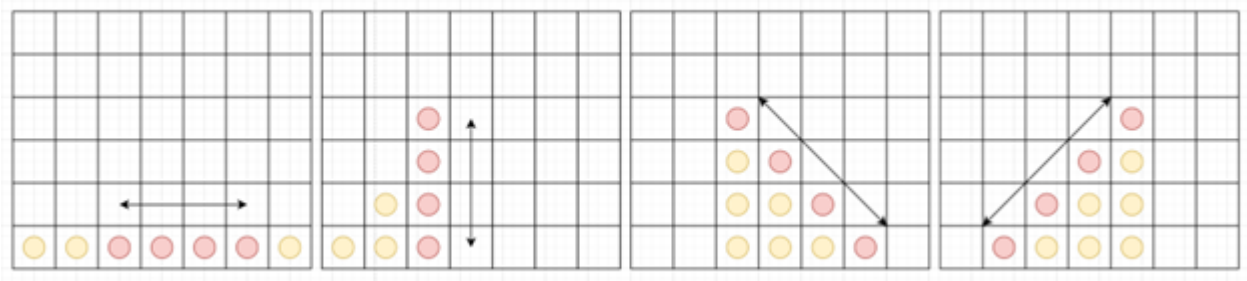

Figure 1. Ways to win a Connect Four

Connect Four (also a game we might have learned as a child) is also a solved game, but unlike Tic-Tac-Toe the number of possible combinations of tokens is rather large. The game space is approximately 4,531,985,219,092 (about 4.5 trillion) different combinations that can be created with a basic board of 6x7 (42 playing pieces). As the board expands, so does the number of combinations.

There are a few variations of the game that will require the agent to maximize its playing skills, such as an expanding board size (both horizontally and vertically), as well as the number of consecutive tokens needed to win. Your agent will also play in a multiple player version that pits your agent against two opponents (three players at one time), as well as the ultimate version, a three-player mode where the owner of your opponents' tokens will be hidden from view; your agent will know that there are pieces in certain positions but will not know which token belongs to which player, other than its own.

Connect Four Game Types

In ***Connect Four Basic***, the agent will play the normal version of the game. The game board will be a standard board of 6 rows and 7 columns. To win an agent must get 4 of its tokens consecutively placed on the board in either a horizontal, vertical or diagonal direction.

In ***Connect Four Extended***, the agent will play a variety of games where the board width (columns) and the height (rows) will randomly increase in size from the standard board size. The number of sequential tokens needed to win a game will also increase (depending on the board size).

In ***Connect Four Multiplayer***, the agent will play games on a fixed sized board against 2 other agents (3 in total). The rules of the game are the same as basic with the agent required to get 4 consecutive tokens in a row to win a game.

In ***Connect Four Hidden Multiplayer***, the agent plays against 2 other agents, but the other agents' tokens color will be 'hidden' from your agent. Your agent will be able to "see" the other agents' tokens, but their colors are masked. The agent will be able to distinguish their own, but there will be no indication of which of the other tokens belong to which agent.

The Code

To make it easier to start the project and focus on the concepts involved (rather than the nuts and bolts of controlling how the game is played), you'll be working from an agent framework in Python. To get started, download the KBAI Gaming Starter Code as a zip file from the Files section in Canvas.

You will code your agent inside the `make_move` method contained in the GameAgent.py file. The method will take as input the current state of a game board—including what type of game and the current moves that have been played—and return the next move to make. For Tic-Tac-Toe, the agent should return a tuple

representing where to move (e.g. (0,0) for the top left, (2,2) for the bottom right, (2,0) for the bottom left. For Connect 4, the agent should return only the index of the column in which to drop a token; the token will automatically "drop" to the lowest available spot.

If your agent makes an invalid move at any time during the game, the game is halted, and your agent loses that game. A move is invalid if it is outside the scope of the board (e.g. playing in row 4 of Tic-Tac-Toe, or playing in column 9 of Connect Four Basic) or if the spot in which the move is made is already taken (e.g. playing (0, 0) when that spot is filled in Tic-Tac-Toe, or selecting column 4 in Connect 4 Basic when that column is already filled with tokens).

You may also create any additional classes, methods, and/or .py files needed to organize your code. `make_move` is the entry point for the agent, and all other classes/files in the starter code zip file are for the game mechanics and are not to be submitted with your agent code to the autograder; the Gradescope autograder contains its own version of these files so only the GameAgent.py and any other supporting files you create need to be submitted.

As part of the starter code, you are given an opponent to play against. This opponent will play purely random moves; you can use this opponent to test your agent's ability to play through a full game. We also encourage you to have your agent play against itself as another way to test out its intelligence.

Working with the Code

The framework code is available in the Files section of Canvas under the Starter Code directory. It consists of the following 2 primary files:

- GameAgent.py – the primary agent that you will modify
- GameDriver.py – handles running the games your agent will compete in

There are other support files that are needed for running the game. These files should not be modified as there are similar versions that will be used when your agent is submitted to the Autograder. These files currently consist of:

- Game.py – this is the primary object that is passed to your agent during the game, it will contain the type of game being played, the current game board and a few other bits of information that you might find useful when designing your agent
- GameEndingStatus.py – the states a game could end in
- GameStats.py – some statistical items for a game: who won, final board, player’s moves etc.
- GameUtil.py – some helper methods that will print board states and the Game Stats
- Token.py – will represent a player’s token in a game. Your agent will get instantiated with one of them, so you know what your player token is

There are also 2 game controllers: one for Tic-Tic-Toe and the other for Connect Four. These are the primary controllers that play the games. There is no need to modify these files as they will be replaced in the Autograder when Gradescope is run so any local modifications will not be useful other than for local testing.

You may explore all the files in the package but only need to modify the GameAgent.py file as this will be the file you will upload to Gradescope.

For Tic-Tac-Toe your agent should return a set of coordinates in the form of a tuple of integers. The first number corresponds to the **row** and the second number corresponds to the **column** that your token should

be placed in. On your turn, the game controller will call your agent's make move method and pass it a Game object. Your agent will use this information to select a location on the game board and return that location. If it is a valid location, the board will be updated and passed to the next player. This will continue until one of the agents gets 3 of their tokens in a row, the game board fills up or an invalid move is made. Any invalid move will be an immediate loss for the agent, the game board will be cleared, and the next game will begin.

For Connect Four the game play is a bit different. Unlike Tic-Tac-Toe, the agent needs to only return a single integer corresponding to the **column** the agent's token should be dropped into. In Connect Four the token will be dropped down to the lowest unoccupied position on the board. The premise is basically the same as the primary objective for the agent is to get a specified number of tokens in sequential order. Depending on the game type being played this will change (see the individual assignment instructions for specific game instructions.

Libraries

The permitted libraries for this terms project are:

- The Numpy library (see the requirements.txt for the exact version number) For installation instructions on numpy, see **this page (https://numpy.org/install/)** .
- Any standard library that comes pre-installed with the Python package for the project

Currently (as of this writing) we are using Python 3.13 for our Autograder.

Submitting to Gradescope

Once you have developed an agent that can play one of the games, you may submit to Gradescope to test it against our agents. To get started submitting your code, go to Canvas and click Gradescope on the left sidebar, then 7637. There are five different Gradescope submission assignments for the project, one for each of the five variations of the game (Tic-Tac-Toe, Connect Four Basic, Connect Four Extended, Connect Four Multiplayer, and Connect Four Hidden Multiplayer); when you submit to one variation, Gradescope will run your agent only against that variation. You may develop one agent that can play every variation (by including code in the `make_move` method that selects a strategy based on which game the agent is playing), or you can keep track of your agents yourself and only submit them to the variation they are coded to play.

To submit your code, drag and drop your project files (GameAgent.py and any support .py files you created; you should not submit any of the other files that we have supplied) into the submission window. You may also zip your code up and upload the zip file. You will receive a confirmation message if your submission was successful; otherwise, take the corrective action specified in the error message and resubmit.

When you submit to Gradescope, your agent will play 20 games: 10 games against a Random agent, and 10 games against a Smart agent. Your agent will earn 3 points for each game it wins, and 1 point for each game that ends in a tie. You are limited to 10 total submissions to Gradescope, and 10 minutes of runtime per submission. Your agent is also limited to 6GB of memory.

Points will be awarded according to the following formula:

- 50% of the available participation points if your agent scores at least 25 points against the Random agent.
- 100% of the available participation points if your agent scores at least 15 points against the Smart agent.

- 100% for Tic-Tac-Toe if your agent can Tie at least 80% of the time against the Smart agent.

The Tic-Tac-Toe project is worth up to 5 participation points; each variation of Connect 4 is worth up to 15 participation points.

In addition, after the project deadlines, all agents will be exported and run through a class-wide tournament. The top 10 finishers in that tournament will receive an additional 5 participation points. Tournaments are run as either a double elimination tournament for the 2 player games and a triple elimination for the 3 player games. All agents must use a single file for the agent as we've run into issues with name collisions for external class files.

Deadlines

The projects are due every three weeks starting in week 4. We'll aim to have the tournament before the next deadline. So, the deadlines are:

- Tic-Tac-Toe: Due at the end of Week 4
- Connect 4 Basic: Due at the end of Week 7
- Connect 4 Extended: Due at the end of Week 10
- Connect 4 Multiplayer: Due at the end of Week 13
- Connect 4 Multiplayer Hidden: Due at the end of Week 16